

P3940R0

Differentiate concept tags for C++26: `sender_tag`

Published Proposal, 2025-12-09

Authors:

[Arthur O'Dwyer](#)

[Yi'an Ye](#)

Audience:

WG21

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

The STL has two kinds of tag types: concept tags (traditionally named with `_tag`) and disambiguation tags (traditionally named with `_t`). The new-in-C++26 `<execution>` header introduces four new concept tags, but names them with `_t` instead of `_tag`. We propose to harmonize this before C++26 is released.

Table of Contents

- 1 Introduction**
- 2 Two kinds of tag types**
 - 2.1 Disambiguation tag types
 - 2.2 Concept tag types
 - 2.3 Differences between the two kinds
 - 2.4 Wrong-naming in `<execution>`

3 Additional motivation

4 Proposed wording

- 4.1 [execution.syn]
- 4.2 [exec.sched]
- 4.3 [exec.recv.concepts]
- 4.4 [exec.opstate.general]
- 4.5 [exec.snd.expos]
- 4.6 [exec.snd.concepts]
- 4.7 [exec.connect]
- 4.8 [exec.schedule.from]
- 4.9 [exec.let]
- 4.10 [exec.spawn.future]
- 4.11 [exec.sync.wait]
- 4.12 [exec.spawn]
- 4.13 [exec.cmplsig]
- 4.14 [exec.as.awaitable]
- 4.15 [exec.inline.scheduler]
- 4.16 [exec.task.scheduler]
- 4.17 [task.class]
- 4.18 [task.state]
- 4.19 [exec.counting.scopes.general]

§ 1. Introduction

This paper is condensed from Arthur O'Dwyer's blog post "[Two kinds of tag types: `foo\_t` and `foo\_tag`](#)" (2025-12-03). However, the impetus for that blog post and for this proposal itself was identified by Yi'an Ye in the std-proposals thread "[\[execution\] sender_t or sender_tag?](#)" (2025-11-10).

§ 2. Two kinds of tag types

In C++, when we have a type that carries no data — whose only "identity" is its type — we conventionally call that a "tag type." When you see code like `struct X {}`; it's often the case that `X` is a tag type. However, not all tag types are created equal: there are at least two major disjoint use-

cases for tag types, and the STL (as of 2025) therefore uses two distinct naming conventions for their identifiers.

There is no widely recognized nomenclature for these two kinds of tag types, as far as I know, so I'm going to call them *disambiguation* tags and *concept* tags.

§ 2.1. Disambiguation tag types

A disambiguation tag type looks like this:

```
struct foo_t {}; // maybe with an explicit ctor
inline constexpr foo_t foo = foo_t();
```

A disambiguation tag is passed explicitly by the caller to select among overloads of a library function — typically, a constructor. (Because otherwise we could use the function name itself to disambiguate; but all constructors belong to the same overload set by definition.) So we end up with code like this:

```
struct useful_class {
    explicit useful_class(foo_t, int, int);
    explicit useful_class(bar_t, int, int);
};

auto myVar = useful_class(foo, 1, 2);
```

In this pattern the tag's name (sans `_t`) denotes a verb or prepositional phrase, or some kind of description of the functionality you're asking for — a description that we *would* have put into the name of the function, except that we can't, because the function is a constructor. Examples from the STL include:

- `std::defer_lock_t`
- `std::from_range_t`
- `std::in_place_t` (which [used to be weirder, but was fixed](#))
- `std::piecewise_construct_t`
- `std::unexpected_t`

All of the above are passed only to constructors. `std::nothrow_t` is passed to `operator new`. `std::destroying_delete_t` is passed to `operator delete`.

§ 2.2. Concept tag types

A concept tag type looks like this instead:

```
struct foo_tag {};  
  
template<class T> concept foo = ~~~~;
```

A concept tag is *never* named explicitly at the call-site. It is used in generic programming — that is, inside a template that doesn't know quite all the capabilities of the type it's received. So the template will dispatch on a member typedef with a name conventionally ending in `_category` or `_concept`, which the user-supplied type must provide. Like this:

```
template<class T> void internal_algorithm(T, foo_tag);  
template<class T> void internal_algorithm(T, bar_tag);  
  
template<class T>  
void useful_template(T t) {  
    internal_algorithm(t, typename T::your_category());  
}  
  
struct MyFoolikeClass {  
    using your_category = foo_tag;  
};
```

You can combine this pattern with inheritance, to build a class hierarchy of concept tags corresponding to your concept hierarchy of concepts. Then my user-supplied class `U` should (or is assumed to) model your `concept foo` exactly when `U::your_category` is — *or is derived from* — `foo_tag`. [The STL does it:](#)

```
struct forward_iterator_tag : input_iterator_tag {};  
  
template<class I>  
concept forward_iterator =  
    input_iterator<I> &&  
    derived_from<ITER_CONCEPT(I), forward_iterator_tag> &&  
    ~~~~;
```

In this pattern the tag's name (sans `_tag`) always denotes a concept — a category of behaviors that the user-supplied type has — and the member typedef name always ends in `_category` or

`_concept`. The C++23 STL has only one family of examples:

- `iterator_category` and `iterator_concept` (`input_iterator_tag`, `output_iterator_tag`, `forward_iterator_tag`, etc.)

But Boost has many more:

- `directed_category` (`directed_tag`, `undirected_tag`, etc.) in [Boost.Graph](#)
- `traversal_category` (`vertex_list_graph_tag`, etc.) in [Boost.Graph](#)
- `fusion_tag` (`deque_tag`, `vector_tag`, etc.) in [Boost.Fusion](#)
- `stepper_category` (`stepper_tag`, etc.) in [Boost.Numeric.Odeint](#)
- `orientation_category` (`row_major_tag`, `column_major_tag`, etc.) in [Boost.uBlas](#)
- `type_category` (`tensor_tag`, etc.) in [Boost.uBlas](#)
- `storage_category` (`dense_tag`, `packed_tag`, `sparse_tag`, etc.) in [Boost.uBlas](#)

§ 2.3. Differences between the two kinds

Disambiguation tags	Concept tags
Ends with <code>_t</code>	Ends with <code>_tag</code>
Without the suffix it's an inline constexpr variable	Without the suffix it's a concept
Used in concrete programming	Used only in generic programming
No associated member typedef	Associated member typedef ending with <code>_category</code> or <code>_concept</code>
Named in the caller	Named in the typedef definition, and in the callee
Callee is usually a constructor	Callee is generic-programming machinery
Never derive from each other	May derive from each other to form a hierarchy
No associated concept	Concept often involves <code>derived_from</code>

§ 2.4. Wrong-naming in `<execution>`

The `<execution>` header (added in [P2300](#)) adds four new "concept tags," following the pattern precisely to the letter, *except* that (as of December 2025) it misnames them using `_t` rather than `_tag`. In each case we have the tag type and its associated concept:

```
struct sender_t {};  
  
template<class Sndr>  
    concept sender = ~~~~;
```

User code (e.g. this example from [\[exec.cmplsig\]/2](#)) has a member typedef ending in `_concept`:

```
struct my_sender {  
    using sender_concept = sender_t;  
    ~~~~  
};
```

This follows the "concept tag" pattern to the letter... except for the naming of `sender_t`!

Property of <code>sender_t</code>	Disambiguation tag-like	Concept tag-like
Ends with <code>_t</code>	✓	✗
Without the suffix it's a concept	✗	✓
Used only in generic programming	✗	✓
Associated member typedef ending with <code>_concept</code>	✗	✓
Named in the typedef definition only	✗	✓
Callee is generic-programming machinery	✗	✓
No derived classes in the Standard today	✓	✓
Concept involves <code>derived_from</code>	✗	✓

So `sender_t` really "should" have been named `sender_tag`, as it has been (organically and independently, as far as we know) in [jaredhoberock/croquet](#) since 2019, [janciesko/stdexx](#) since June 2025, [Cra3z/coio](#) since August 2025, and maybe a few other places.

In an ideal world, the following four "concept tag" types would be renamed before we finalize C++26:

- `sender_t`, associated with `sender_concept` and `concept sender`
- `receiver_t`, associated with `receiver_concept` and `concept receiver`
- `operation_state_t`, associated with `operation_state_concept` and `concept operation_state`
- `scheduler_t`, associated with `scheduler_concept` and `concept scheduler`

§ 3. Additional motivation

While writing up [\[exec.schedule.from\]](#), we noticed that the type name `receiver_t` is shadowed in that section. The local alias `receiver_t` is an alias for `receiver_type`, which has a member `receiver_concept` aliased to `receiver_t`. But this second use of `receiver_t` is not the `receiver_t` we're in the middle of defining; it's the global concept tag! This is not *formally* an ambiguity in the spec, but it's not very felicitous. Our proposed change eliminates the notional overloading by cleanly separating the global concept tag `receiver_tag` from the local concrete type `receiver_t`.

Yi'an predicts that future standards will define not only `std::execution::scheduler_tag`, but further derived tags such as `std::execution::network_scheduler_tag`, `std::execution::pipe_scheduler_tag`, etc., all derived from `scheduler_tag` and indicating concepts that subsume `scheduler`. That is, we predict that the future evolution of C++ will make `scheduler_tag` conform even more tightly to the "concept tag" pattern than it does today.

§ 4. Proposed wording

§ 4.1. [\[execution.syn\]](#)

Modify [\[execution.syn\]](#) as follows. Note that the names of query object types such as `set_value_t` and `start_t` are not

changed; only the names of the four concept tags are changed.

```
[...]
// [exec.sched], schedulers
struct scheduler_tag {};

template<class Sch>
    concept scheduler = see below;

// [exec.recv], receivers
struct receiver_tag {};

template<class Rcvr>
    concept receiver = see below;

template<class Rcvr, class Completions>
    concept receiver_of = see below;

struct set_value_t { unspecified };
struct set_error_t { unspecified };
struct set_stopped_t { unspecified };

inline constexpr set_value_t set_value{};
inline constexpr set_error_t set_error{};
inline constexpr set_stopped_t set_stopped{};

// [exec.opstate], operation states
struct operation_state_tag {};

template<class O>
    concept operation_state = see below;

struct start_t;
inline constexpr start_t start{};

// [exec.snd], senders
struct sender_tag {};

template<class Sndr>
    inline constexpr bool enable_sender = see below;

template<class Sndr>
    concept sender = see below;
[...]
```

§ 4.2. [exec.sched]

Modify [\[exec.sched\]](#) as follows:

1. The `scheduler` concept defines the requirements of a scheduler type ([`exec.async.ops`]). `schedule` is a customization point object that accepts a scheduler. A valid invocation of `schedule` is a schedule-expression.

```
namespace std::execution {
    template<class Sch>
    concept scheduler =
        derived_from<typename remove_cvref_t<Sch>::scheduler_concept, scheduler_tag> &&
        queryable<Sch> &&
        requires(Sch&& sch) {
            { schedule(std::forward<Sch>(sch)) } -> sender;
            { auto(get_completion_scheduler<set_value_t>(
                get_env(schedule(std::forward<Sch>(sch))))) }
                -> same_as<remove_cvref_t<Sch>>;
        } &&
        equality_comparable<remove_cvref_t<Sch>> &&
        copyable<remove_cvref_t<Sch>>;
}
[...]
```

§ 4.3. [`exec.recv.concepts`]

Modify [`exec.recv.concepts`] as follows:

1. A receiver represents the continuation of an asynchronous operation. The `receiver` concept defines the requirements for a receiver type ([`exec.async.ops`]). The `receiver_of` concept defines the requirements for a receiver type that is usable as the first argument of a set of completion operations corresponding to a set of completion signatures. The `get_env` customization point object is used to access a receiver's associated environment.

```
namespace std::execution {
    template<class Rcvr>
    concept receiver =
        derived_from<typename remove_cvref_t<Rcvr>::receiver_concept, receiver_tag> &&
        requires(const remove_cvref_t<Rcvr>& rcvr) {
            { get_env(rcvr) } -> queryable;
        } &&
        move_constructible<remove_cvref_t<Rcvr>> && // rvalues are movable, and
        constructible_from<remove_cvref_t<Rcvr>, Rcvr>; // lvalues are copyable
}
[...]
```

§ 4.4. [`exec.opstate.general`]

Modify [`exec.opstate.general`] as follows:

1. The `operation_state` concept defines the requirements of an operation state type ([`exec.async.ops`]).

```
namespace std::execution {
    template<class O>
        concept operation_state =
            derived_from<typename O::operation_state_concept, operation_state_t> &&
            requires (O& o) {
                start(o);
            };
}
[...]
```

§ 4.5. [`exec.snd.expos`]

Modify [`exec.snd.expos`] as follows:

```
[...]

template<class Sndr, class Rcvr, class Index>
    requires valid-specialization<env-type, Index, Sndr, Rcvr>
    struct basic-receiver { // exposition only
        using receiver_concept = receiver_tag;
    }

[...]

template<class Sndr, class Rcvr>
    requires valid-specialization<state-type, Sndr, Rcvr> &&
           valid-specialization<connect-all-result, Sndr, Rcvr>
    struct basic-operation : basic-state<Sndr, Rcvr> { // exposition only
        using operation_state_concept = operation_state_tag;
    }

[...]

template<class Tag, class Data, class... Child>
    struct basic-sender : product-type<Tag, Data, Child...> { // exposition only
        using sender_concept = sender_tag;
    }

[...]

    struct not-a-sender {
        using sender_concept = sender_tag;
    }

[...]
```

§ 4.6. [`exec.snd.concepts`]

Modify [`exec.snd.concepts`] as follows:

```
[...]
template<class Sndr>
    concept is-sender = // exposition only
        derived_from<typename Sndr::sender_concept, sender_tag>;
[...]
```

§ 4.7. [exec.connect]

Modify the example in [exec.connect] as follows:

4. Let *operation-state-task* be the following exposition-only class:

```
namespace std::execution {
    struct operation-state-task { // exposition only
        using operation_state_concept = operation_state_tag;
    }
[...]
```

§ 4.8. [exec.schedule.from]

Modify [exec.schedule.from] as follows. Note that the name of the local type *receiver_t* in paragraph 6 is not changed; only the name of the global concept tag *receiver_t* in paragraph 10 is changed.

6. The member *impls-for*<schedule_from_t>::*get-state* is initialized with a callable object equivalent to the following lambda:

```
[<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(see below)
    requires sender_in<child-type<Sndr>, FWD-ENV-T(env_of_t<Rcvr>)> {

    auto& [_, sch, child] = sndr;

    using sched_t = decltype(auto(sch));
    using variant_t = see below;
    using receiver_t = see below;
    using operation_t = connect_result_t<schedule_result_t<sched_t>, receiver_t>;
    constexpr bool nothrow = noexcept(connect(schedule(sch), receiver_t{nullptr}));

[...]
```

10. *receiver_t* is an alias for the following exposition-only class:

```
namespace std::execution {
    struct receiver-type {
        using receiver_concept = receiver_tag;
        state-type* state; // exposition only
    }
[...]
```

§ 4.9. [exec.let]

Modify [exec.let] as follows:

6. Let *receiver2* denote the following exposition-only class template:

```
namespace std::execution {
    template<class Rcvr, class Env>
    struct receiver2 {
        using receiver_concept = receiver_tag;
    }
    [...]
}
```

§ 4.10. [exec.spawn.future]

Modify [exec.spawn.future] as follows:

5. Let *spawn-future-receiver* be the exposition-only class template:

```
namespace std::execution {
    template<class Completions>
    struct spawn-future-receiver {
        using receiver_concept = receiver_tag;
    }
    [...]
}
```

// exposition only

§ 4.11. [exec.sync.wait]

Modify [exec.sync.wait] as follows:

```
[...]
template<class Sndr>
struct sync-wait-receiver {
    using receiver_concept = execution::receiver_tag;
}
[...]
```

// exposition only

§ 4.12. [exec.spawn]

Modify [exec.spawn] as follows:

```
[...]
struct spawn-receiver {
    using receiver_concept = receiver_tag;
}
[...]
```

// exposition only

§ 4.13. [exec.cmplsig]

Modify the example in [exec.cmplsig] as follows:

[Example 1:

```
struct my_sender {
    using sender_concept = sender_tag;
    using completion_signatures =
        execution::completion_signatures<
            set_value_t(),
            set_value_t(int, float),
            set_error_t(exception_ptr),
            set_error_t(error_code),
            set_stopped_t()>;
};
```

Declares my_sender to be a sender that [...] —end example]

§ 4.14. [exec.as.awaitable]

Modify [exec.as.awaitable] as follows:

3. *awaitable-receiver* is equivalent to:

```
struct awaitable-receiver {
    using receiver_concept = receiver_tag;
    variant<monostate, result-type, exception_ptr>* result_ptr;    // exposition only
    coroutine_handle<Promise> continuation;                      // exposition only
    // see below
};
```

§ 4.15. [exec.inline.scheduler]

Modify [exec.inline.scheduler] as follows:

```
namespace std::execution {
    class inline_scheduler {
        class inline_sender;                                // exposition only

        template<receiver R>
            class inline_state;                             // exposition only

    public:
        using scheduler_concept = scheduler_tag;

        constexpr inline_sender schedule() noexcept { return {}; }
        constexpr bool operator==(const inline_scheduler&) const noexcept = default;
    };
}
```

§ 4.16. [exec.task.scheduler]

Modify [exec.task.scheduler] as follows:

```
class task_scheduler {
    class ts_sender;                // exposition only

    template<receiver R>
        class state;                // exposition only

public:
    using scheduler_concept = scheduler_tag;

    template<class Sch, class Allocator = allocator<void>>
        requires (!same_as<task_scheduler, remove_cvref_t<Sch>>)
            && scheduler<Sch>
    explicit task_scheduler(Sch&& sch, Allocator alloc = {});

    ts_sender schedule();

    friend bool operator==(const task_scheduler& lhs, const task_scheduler& rhs)
        noexcept;
    template<class Sch>
        requires (!same_as<task_scheduler, Sch>)
            && scheduler<Sch>
    friend bool operator==(const task_scheduler& lhs, const Sch& rhs) noexcept;

private:
    shared_ptr<void> sch_; // exposition only
};

[...]
```

```
class task_scheduler::ts_sender {    // exposition only
public:
    using sender_concept = sender_tag;

    template<receiver Rcvr>
        state<Rcvr> connect(Rcvr&& rcvr);
};

[...]
```

```
template<receiver R>
class task_scheduler::state {        // exposition only
public:
    using operation_state_concept = operation_state_tag;

    void start() & noexcept;
};
```

§ 4.17. [task.class]

Modify [task.class] as follows:

```
namespace std::execution {
    template<class T, class Environment>
    class task {
        // [task.state]
        template<receiver Rcvr>
        class state;                                // exposition only

    public:
        using sender_concept = sender_tag;
        using completion_signatures = see below;
    [...]
}
```

§ 4.18. [task.state]

Modify [task.state] as follows:

```
namespace std::execution {
    template<class T, class Environment>
    template<receiver Rcvr>
    class task<T, Environment>::state {            // exposition only
    public:
        using operation_state_concept = operation_state_tag;
    [...]
}
```

§ 4.19. [exec.counting.scopes.general]

Modify [exec.counting.scopes.general] as follows:

```
[...]
template<>
struct impls-for<scope-join-t> : default-impls {
    template<class Scope, class Rcvr>
    struct state {                                // exposition only
        struct rcvr-t {                            // exposition only
            using receiver_concept = receiver_tag;
        }
    [...]
}
```